System Architecture Design

Stefano Buglione, Lauryn Brown, Marie Schwarz, Nicolas Fernandez

Professor David Gaitros

CEN4090L: Software Engineering Capstone Project

Florida State University

1. Architecture Goals and Constraints

- Provide educational answers about personalized medicine without offering diagnosis or treatment.
- Ensure transparency through citations from validated and peer-reviewed sources.
- Reduce hallucinations and bias using Retrieval-Augmented Generation (RAG), guardrails, and disclaimers.
- Provide a web-based application with avatar visualization, text responses, and optional text-to-speech.

2. High-Level System Architecture

- Client Layer: Web UI with chat interface, avatar renderer, audio controls, and optional speech-to-text.
- Backend Layer: API services for chat processing, guardrails, retrieval, LLM generation, citations, and feedback.
- Knowledge Layer: Curated medical corpus stored in document storage and vector databases for semantic search.
- Observability Layer: Logging, metrics, uptime monitoring, and evaluation pipelines.

3. Main System Components

- Frontend (Browser): Chat interface, avatar state controller (Idle, Loading, Speaking), audio playback module, feedback widget.
- Backend Server: API Gateway, conversation orchestrator, policy and guardrails engine, retrieval service, LLM generator, citation formatter, text-to-speech service.
- Data Stores: Vector database for embeddings, document store for source material, feedback database, and system logs.

4. Data Flow

- 1. User submits a prompt through the web chat interface.
- 2. The frontend sends the request to the backend API.
- 3. Guardrails check if the prompt violates scope (diagnosis/treatment).
- 4. Retrieval service searches peer-reviewed knowledge sources using embeddings.
- 5. LLM generates a response grounded in retrieved information.
- 6. Citation service attaches relevant source references.
- 7. Response is returned to the UI and optionally converted to audio through the TTS service.
- 8. User may submit feedback after the interaction.

5. Deployment Architecture

- Browser client hosting the user interface.
- Backend API server handling orchestration and AI logic.
- Vector database for semantic retrieval.
- Document storage for validated medical sources.
- Feedback database for user interaction data.

6. Security, Privacy, and Safety Controls

- Minimal storage of user data with anonymized session IDs.
- Encryption and redaction if sensitive data is stored.
- Guardrails to prevent medical diagnosis or treatment recommendations.
- Use of validated sources to minimize hallucination risks.

7. Requirement Traceability

- Chat interaction requirement => Chat UI and Conversation Orchestrator.
- Educational-only responses => Policy and Guardrails Engine.
- Citations requirement => Retrieval and Citation Services.
- Avatar and audio interaction => Avatar Controller and TTS Service.
- Performance requirements => Monitoring and scalable backend architecture.